

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	R. Uday Kumar reddy	Reg. No.:	192424190
Section / Batch:	Slot: C	Date:	20/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Longest Palindromic Subsequence

Given a string, find the length of the longest subsequence that is also a palindrome. The subsequence need not be contiguous but must maintain the original order. The objective is to determine the maximum possible palindrome length. Use Dynamic Programming to compute the result efficiently.

Input Format:

s = input string

Sample Input:

s = BBABCBCAB

Expected Output:

Length = 7

Code of Conduct

/ R. Uday kumar Reddy (192424190)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R.UDAY_____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. AIM

To find the **length of the Longest Palindromic Subsequence (LPS)** of a given string using the **Dynamic Programming** technique. The subsequence need not be contiguous, but the original order of characters must be maintained.

2. PROBLEM

Given a string, find the longest subsequence that is also a palindrome.

For example:

Input:

BBABCBCAB

A longest palindromic subsequence is:

BABCBAB

Therefore:

Length = 7

The objective is to efficiently determine the maximum possible palindrome length using Dynamic Programming.

3. PSEUDOCODE

START

Read string S

$N \leftarrow \text{length of } S$

Create DP[N][N]

FOR i = 0 to N-1

$DP[i][i] \leftarrow 1$

END FOR

FOR length = 2 to N

 FOR i = 0 to N - length

```

    j ← i + length - 1
    IF S[i] = S[j] THEN
        IF length = 2 THEN
            DP[i][j] ← 2
        ELSE
            DP[i][j] ← DP[i+1][j-1] + 2
        END IF
    ELSE
        DP[i][j] ← MAX(DP[i+1][j],
            DP[i][j-1])
    END IF

END IF

END FOR

END FOR

Print DP[0][N-1]

STOP

```

4. ALGORITHM

Step 1: Read the input string S.

Step 2: Find the length N of the string.

Step 3: Create a two-dimensional DP table DP[N][N].

Step 4: Initialize the diagonal elements to 1, because every single character is a palindrome.

$DP[i][i] = 1$

Step 5: Consider substrings of length 2 to N.

Step 6: Compare the first and last characters of each substring.

Step 7: If both characters are equal:

$$DP[i][j] = DP[i+1][j-1] + 2$$

Step 8: If the characters are different:

$$DP[i][j] = \max(DP[i+1][j], DP[i][j-1])$$

Step 9: Continue filling the DP table until the complete string is processed.

Step 10: The value $DP[0][N-1]$ gives the length of the Longest Palindromic Subsequence.

5. MANUAL PROCESS

Given String

$S = BBABCBCAB$

Write the characters with their positions:

Position : 0 1 2 3 4 5 6 7 8

Character: B B A B C B C A B

Step 1: Single Characters

Every single character is a palindrome of length 1.

$B \rightarrow 1$

$B \rightarrow 1$

$A \rightarrow 1$

$B \rightarrow 1$

$C \rightarrow 1$

$B \rightarrow 1$

$C \rightarrow 1$

$A \rightarrow 1$

$B \rightarrow 1$

Therefore:

$$DP[i][i] = 1$$

Step 2: Substrings of Length 2

Compare two characters at a time.

$$BB \rightarrow B = B \rightarrow 2$$

$$BA \rightarrow B \neq A \rightarrow 1$$

$$AB \rightarrow A \neq B \rightarrow 1$$

$$BC \rightarrow B \neq C \rightarrow 1$$

$$CB \rightarrow C \neq B \rightarrow 1$$

$$BC \rightarrow B \neq C \rightarrow 1$$

$$CB \rightarrow C \neq B \rightarrow 1$$

$$AB \rightarrow A \neq B \rightarrow 1$$

So:

$$BB = 2$$

$$BA = 1$$

$$AB = 1$$

$$BC = 1$$

$$CB = 1$$

$$BC = 1$$

$$CB = 1$$

$$AB = 1$$

Step 3: Substrings of Length 3

Consider:

$$BBA$$

BAB

ABC

BCB

CBC

BCB

BCA

CAB

Example: BBA

$B \neq A$

So:

$DP = \max(BB, BA)$

$= \max(2, 1)$

$= 2$

Therefore:

$BBA \rightarrow 2$

Example: BAB

$B = B$

Therefore:

$DP = 1 + 2$

$= 3$

So:

$BAB \rightarrow 3$

Similarly:

$BCB \rightarrow 3$

CBC → 3

Step 4: Continue the Same Process

For every larger substring, apply the same rules:

If first character = last character

$$DP[i][j]$$

↓

$$DP[i+1][j-1] + 2$$

If they are different:

If first character ≠ last character

$$DP[i][j]$$

↓

$$\max(DP[i+1][j], DP[i][j-1])$$

Step 5: Final DP Table

	B	B	A	B	C	B	C	A	B
B		1	2	2	3	3	5	5	7
B			1	1	3	3	5	5	7
A				1	1	1	3	3	5
B					1	1	3	3	5
C						1	1	3	3
B							1	1	3
C								1	1
A									1
B									

The final value is:

$$\text{DP}[0][8] = 7$$

Therefore:

Longest Palindromic Subsequence = 7

One possible subsequence is:

BABCBAB

Check:

Forward : B A B C B A B

Reverse : B A B C B A B

Hence, BABCBAB is a palindrome of length 7.

6. Python code:

```
s = input("Enter string: ")
```

```
n = len(s)
```

```
dp = [[0] * n for _ in range(n)]
```

```
print("\n=====
=====")
```

```
print("|| LONGEST PALINDROMIC SUBSEQUENCE ||")
```

```
print("=====
```

```
print("\n Input String :", s)
```

```
print(" Length      :", n)
```

Step 1

```
print("\n _____ STEP 1 : INITIALIZATION _____")
```

```
print(" | Every single character is a palindrome of length 1 | ")
```

```
print(" L _____  
_____l ")
```

```
for i in range(n):
```

```
    dp[i][i] = 1
```

```
print(" ", end="")
```

```
for ch in s:
```

```
    print(ch, end=" ")
```

```
print()
```

```
print(" ", end="")
```

```
for i in range(n):
```

```
    print("l", end=" ")
```

```
print()
```

```
# Step 2
```

```
print("\n ┌── STEP 2 : BUILDING DP TABLE ───────────────────┐")
```

```
for length in range(2, n + 1):
```

```
    print(" | ")
```

```
    print(" | Checking length =", length)
```

```
    for i in range(n - length + 1):
```

```
        j = i + length - 1
```

```
        part = s[i:j + 1]
```

```
        if s[i] == s[j]:
```

```
            if length == 2:
```

```
                dp[i][j] = 2
```

```
            else:
```

```
                dp[i][j] = dp[i + 1][j - 1] + 2
```

```

        print(" | ", part, "→", s[i],
              "=", s[j], "✓ MATCH →", dp[i][j])
    else:
        dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
        print(" | ", part,
              "→", s[i], "≠", s[j],
              "✗ SELECT →", dp[i][j])

print("┌───────────────────────────────────────────────────────────────────────────────────┐")
# Step 3
print("\n└── STEP 3 : FINAL DP TABLE ───────────────────────────────────────────────────┘")
print(" | ", end="")

for ch in s:
    print(ch, end=" ")

print()

print(" | " + "-" * (n * 4))

for i in range(n):
    print(" | " + s[i] + " | ", end="")

    for j in range(n):
        if j < i:
            print(" ", end=" ")

        print(dp[i][j], end=" ")

    else:
        print(dp[i][j], end=" ")

```

```
print("_____")
```

```
print("\n _____ STEP 4 : RECONSTRUCTING PALINDROME _____")
```

$$j = n - 1$$

```
right = ""
```

```
if s[i] == s[j]:
```

```
left += s[i]
```

```
right = s[j] + right
```

$$j = 1$$

```
print(" | ", s[i], "→", s[j],
```

```
i += 1
```

```
print(" | ", s[i], "→", s[j],
```

" → MOVE RIGHT"

```

[ Console | I/O | AI Agent ]
Enter string: bbabcbcab

LONGEST PALINDROMIC SUBSEQUENCE

Input String : bbabcbcab
Length       : 9

STEP 1 : INITIALIZATION
Every single character is a palindrome of length 1

b   b   a   b   c   b   c   a   b
1   1   1   1   1   1   1   1   1

STEP 2 : BUILDING DP TABLE

Checking length = 2
bb → b = b ✓ MATCH → 2
ba → b ≠ a X SELECT → 1
ab → a ≠ b X SELECT → 1
bc → b ≠ c X SELECT → 1
cb → c ≠ b X SELECT → 1
bc → b ≠ c X SELECT → 1
ca → c ≠ a X SELECT → 1
ab → a ≠ b X SELECT → 1

```

Checking length = 3

bba	→ b ≠ a	X	SELECT → 2
bab	→ b = b	✓	MATCH → 3
abc	→ a ≠ c	X	SELECT → 1
bc b	→ b = b	✓	MATCH → 3
c b c	→ c = c	✓	MATCH → 3
bca	→ b ≠ a	X	SELECT → 1
cab	→ c ≠ b	X	SELECT → 1

Checking length = 4

bbab	→ b = b	✓	MATCH → 3
bab c	→ b ≠ c	X	SELECT → 3
ab c b	→ a ≠ b	X	SELECT → 3
b c b c	→ b ≠ c	X	SELECT → 3
c b c a	→ c ≠ a	X	SELECT → 3
b c a b	→ b = b	✓	MATCH → 3

Checking length = 5

bbabc	→ b ≠ c	X	SELECT → 3
bab c b	→ b = b	✓	MATCH → 3
ab c b c	→ a ≠ c	X	SELECT → 3
b c b c a	→ b ≠ a	X	SELECT → 3
c b c a b	→ c ≠ b	X	SELECT → 3

Checking length = 6

bbab c b	→ b = b	✓	MATCH → 5
bab c b c	→ b ≠ c	X	SELECT → 3
ab c b c a	→ a = a	✓	MATCH → 5
b c b c a b	→ b = b	✓	MATCH → 5

Checking length = 7

bbab c b c	→ b ≠ c	X	SELECT → 5
bab c b c a	→ b ≠ a	X	SELECT → 5
ab c b c a b	→ a ≠ b	X	SELECT → 5

Checking length = 8

bbab c b c a	→ b ≠ a	X	SELECT → 5
bab c b c a b	→ b = b	✓	MATCH → 7

Checking length = 9

bbab c b c a b	→ b = b	✓	MATCH → 7
----------------	---------	---	-----------

STEP 3 : FINAL DP TABLE

	b	b	a	b	c	b	c	a	b
b	1	2	2	3	3	5	5	5	7
b		1	1	3	3	3	3	5	7
a			1	1	1	3	3	5	5
b				1	1	3	3	3	5
c					1	1	3	3	3
b						1	1	1	3
c							1	1	1
a								1	1
b									1

STEP 4 : RECONSTRUCTING PALINDROME

b ↔ b → TAKE BOTH ✓
 b →/a → MOVE LEFT
 a ↔ a → TAKE BOTH ✓
 b →/c → MOVE LEFT
 c ↔ c → TAKE BOTH ✓
 b ↔ b → TAKE BOTH ✓

FINAL ANSWER

Original String : bbabcbcab
 Palindromic LPS : bacbcab
 Maximum Length : 7